

Área Científica	Programação de Sistemas de Informação
Unidade Curricular	Introdução às Linguagens e Tecnologias Web
Documento	Guião Prático 11: Flask e React
Tema	<i>Views, Templates, Segurança e React</i>

Configuração do Ambiente

Nesta aula utilizaremos o **Flask** (Python) para o backend e **React** para o frontend.

1. **VS Code:** Instale a extensão "Python" da Microsoft.
2. **Python:** Certifique-se de que o Python 3 está instalado.
3. **Flask:** No terminal do VS Code, execute:

```
1 pip install flask
```

1. Estrutura de Views e Templates (Flask)

Fundamento Teórico: O Flask utiliza o padrão decorador para mapear URLs a funções (Views). A função `render_template` da `Jinja2`, que funde o HTML estático com dados dinâmicos.

Tarefa: Crie um ficheiro `app.py` e uma pasta `templates/` com o ficheiro `index.html` dentro.

app.py

```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     contexto = {
8         "titulo": "Bem-vindo à ESTGA",
9         "utilizador": "Catarina",
10        "itens": ["HTML", "CSS", "Flask", "React"]
11    }
12    return render_template('index.html', **contexto)
13
14 if __name__ == '__main__':
```

```

15 app.run(debug=True)

templates/index.html
1 <h1>{{ titulo }}</h1>
2 <p>Ol , {{ utilizador }}!</p>
3 <ul>
4     {% for item in itens %}
5         <li>{{ item }}</li>
6     {% endfor %}
7 </ul>

```

2. Formulários e Proteção CSRF

Fundamento Teórico: Em aplicações reais, o servidor exige um *Token* para validar que a submissão do formulário é legítima. O Flask permite aceder aos dados via `request.form`.

Tarefa: Altere o seu `app.py` para processar um formulário simples.

```

1 from flask import request
2
3 @app.route('/login', methods=['GET', 'POST'])
4 def login():
5     if request.method == 'POST':
6         user = request.form.get('username')
7         # Simula o de verifica o de token
8         token = request.form.get('csrf_token')
9         if token == "SECRET123":
10             return f"Login com sucesso para {user}"
11         return "Erro de Seguran a!", 403
12     return render_template('login.html')

```

3. Introdução ao React: Componentes e Props

Fundamento Teórico: O React baseia-se na composição. *Props* são argumentos passados de componentes pai para filhos, permitindo a reutilização da mesma estrutura com dados diferentes.

Tarefa: Crie um ficheiro `index.html` (fora da pasta `templates`) e utilize a CDN.

```

1 function Saudar(props) {
2     return <h1>Ol , {props.nome}!</h1>;
3 }
4
5 function App() {
6     return (
7         <div>
8             <Saudar nome="Estudante" />

```

```
9         <Saudar nome="Professor" />
10     </div>
11 );
12 }
```

4. Gestão de Estado com useState

Fundamento Teórico: O *State* é a memória de um componente. Quando o estado muda, o React re-renderiza o componente automaticamente.

Tarefa: Implemente um contador que incrementa e decrementa.

```
1 function Contador() {
2     const [valor, setValor] = React.useState(0);
3
4     return (
5         <div>
6             <h2>Contador: {valor}</h2>
7             <button onClick={() => setValor(valor + 1)}>+</button>
8             <button onClick={() => setValor(valor - 1)}>-</button>
9         </div>
10    );
11 }
```

5. Renderização de Listas (Map)

Fundamento Teórico: No React, utilizamos o método `.map()` do JavaScript para transformar uma lista de dados em elementos visuais de forma declarativa.

Tarefa: Crie um componente que recebe uma lista de tarefas e as exibe.

```
1 function ListaTarefas() {
2     const tarefas = ["Estudar Flask", "Praticar React", "Rever CSS"];
3
4     return (
5         <ul>
6             {tarefas.map((t, index) => <li key={index}>{t}</li>)}
7         </ul>
8     );
9 }
```

6. Renderização Condicional

Fundamento Teórico: O React permite decidir o que mostrar com base em condições (operadores ternários ou `<div>`), tornando a interface inteligente.

Tarefa: Crie um componente que mostra uma mensagem apenas se o utilizador estiver "logado".

```
1 function PainelUsuario() {
2   const [logado, setLogado] = React.useState(false);
3
4   return (
5     <div>
6       {logado ? <p>Bem-vindo ao Painel!</p> : <p>Por favor,
7         fa a login.</p>}
8       <button onClick={() => setLogado(!logado)}>
9         {logado ? "Sair" : "Entrar"}
10      </button>
11    </div>
12  );
13 }
```

7. Formulários e Controladores de Input

Fundamento Teórico: Em React, inputs são frequentemente "controlados". Isto significa que o valor do input está ligado ao estado, e qualquer alteração no texto atualiza esse mesmo estado via eventos (e.g., `onChange`).

Tarefa: Crie um formulário simples que permite ao utilizador escrever o seu nome e vê-lo aparecer em tempo real num parágrafo.

```
1 function FormularioNome() {
2   const [nome, setNome] = React.useState("");
3
4   return (
5     <div>
6       <input
7         type="text"
8         placeholder="Escreva o seu nome"
9         value={nome}
10        onChange={(e) => setNome(e.target.value)}
11      />
12       <p>O nome digitado : {nome}</p>
13     </div>
14   );
15 }
```

8. O Hook `useEffect` e Efeitos Secundários

Fundamento Teórico: O `useEffect` serve para lidar com ações fora do ciclo de renderização puro, como pedidos a APIs, subscrições ou manipulação manual do DOM após o componente aparecer no ecrã.

Tarefa: Implemente um componente que exibe um cronómetro que aumenta automaticamente a cada segundo.

```

1 function Cronometro() {
2   const [segundos, setSegundos] = React.useState(0);
3
4   React.useEffect(() => {
5     const intervalo = setInterval(() => {
6       setSegundos(s => s + 1);
7     }, 1000);
8
9     return () => clearInterval(intervalo); // Limpeza ao
      desmontar
10  }, []);
11
12  return <h1>Tempo decorrido: {segundos}s</h1>;
13 }

```

9. Composição: A Propriedade children

Fundamento Teórico: O React permite passar componentes ou HTML como "conteúdo" para dentro de outros componentes através da prop especial `children`. Isto é ideal para criar molduras, modais ou *layouts* genéricos.

Tarefa: Crie um componente `Cartao` que adiciona uma borda e sombra a qualquer conteúdo que seja colocado dentro dele.

```

1 function Cartao(props) {
2   const estilo = {
3     border: "1px solid #ccc",
4     padding: "20px",
5     borderRadius: "8px",
6     boxShadow: "2px 2px 10px rgba(0,0,0,0.1)",
7     margin: "10px"
8   };
9
10  return (
11    <div style={estilo}>
12      {props.children}
13    </div>
14  );
15 }
16
17 function App() {
18   return (
19     <Cartao>
20       <h2>Título do Cartão</h2>
21       <p>Este conteúdo está dentro do componente genérico</p>
22     </Cartao>

```

```
23         );  
24     }
```

10. Flask como API (JSON)

Fundamento Teórico: Em aplicações modernas (SPA), o backend muitas vezes não renderiza HTML, mas sim dados puros em formato JSON. O Flask utiliza a função `jsonify` para este propósito.

Tarefa: Adicione um novo *endpoint* ao seu `app.py` que retorne uma lista de produtos em formato JSON.

```
1 from flask import jsonify
2
3 @app.route('/api/produtos')
4 def get_produtos():
5     produtos = [
6         {"id": 1, "nome": "Teclado Mecânico", "preco": 85.0},
7         {"id": 2, "nome": "Rato Gaming", "preco": 45.0},
8         {"id": 3, "nome": "Monitor 4K", "preco": 299.0}
9     ]
10    return jsonify(produtos)
```

11. Consumo de APIs com Fetch no React

Fundamento Teórico: Para ligar o frontend ao backend, o React utiliza a API `fetch`. Geralmente, o pedido é feito dentro de um `useEffect` para garantir que os dados são carregados assim que o componente aparece.

Tarefa: Crie um componente `Catalogo` que consome a lista de produtos da sua API Flask e os exibe no ecrã.

```

1 function Catalogo() {
2   const [produtos, setProdutos] = React.useState([]);
3
4   React.useEffect(() => {
5     fetch('http://127.0.0.1:5000/api/produtos')
6       .then(res => res.json())
7       .then(dados => setProdutos(dados))
8       .catch(err => console.error("Erro ao carregar dados:",
9         err));
10
11   }, []);
12
13   return (
14     <div>
15       <h2>0 Nosso Cat logo </h2>
16       <ul>
17         {produtos.map(p => (

```

```
16         <li key={p.id}>{p.nome} - {p.preco}    </li>
17     ))}
18 </ul>
19 </div>
20 );
21 }
```

12. Comunicação entre Componentes (Lifting State Up)

Fundamento Teórico: No React, o estado flui para baixo. Para um componente "filho" alterar algo no "pai", o pai deve passar uma função via *props* que o filho possa chamar.

Tarefa: Crie um componente `FichaProduto` que recebe o nome de um produto e um botão "Comprar". Ao clicar no botão, o componente `App` (pai) deve ser avisado e atualizar um contador de compras.

```
1 function FichaProduto(props) {
2   return (
3     <div style={{border: "1px solid gray", margin: "5px",
4       padding: "5px"}}>
5       <span>{props.nome}</span>
6       <button onClick={props.aoComprar}>Comprar</button>
7     </div>
8   );
9 }
10 function App() {
11   const [carrinho, setCarrinho] = React.useState(0);
12
13   return (
14     <div>
15       <h1>Carrinho: {carrinho} itens</h1>
16       <FichaProduto nome="Laptop" aoComprar={() =>
17         setCarrinho(carrinho + 1)} />
18       <FichaProduto nome="Smartphone" aoComprar={() =>
19         setCarrinho(carrinho + 1)} />
20     </div>
21   );
22 }
```